Sigmoid Activation Function: Differentiation and Single Layer Perceptron

Bivas Kumar Mittra
Registration No: 2021831034

June 21, 2026

1 Differentiating the Sigmoid Activation Function

1.1 Definition

The sigmoid activation function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

1.2 The Derivative

The derivative of the sigmoid function can be expressed in terms of the sigmoid function itself:

$$\boxed{\sigma'(x) = \sigma(x)(1 - \sigma(x))}$$

1.3 Step-by-Step Derivation

Let

$$\sigma(x) = (1 + e^{-x})^{-1}$$

Using the chain rule:

$$\sigma'(x) = -1(1 + e^{-x})^{-2} \cdot \frac{d}{dx}(1 + e^{-x})$$

$$\frac{d}{dx}(1 + e^{-x}) = -e^{-x}$$

Therefore:

$$\sigma'(x) = -1(1 + e^{-x})^{-2} \cdot (-e^{-x})$$

$$\sigma'(x) = \frac{e^{-x}}{(1 + e^{-x})^2}$$

Since $\sigma(x) = \frac{1}{1+e^{-x}}$, we have:

$$1 - \sigma(x) = 1 - \frac{1}{1 + e^{-x}} = \frac{e^{-x}}{1 + e^{-x}}$$

Thus:

$$\sigma(x)(1-\sigma(x)) = \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} = \frac{e^{-x}}{(1+e^{-x})^2}$$

Hence proven:

$$\boxed{\sigma'(x) = \sigma(x)(1-\sigma(x))}$$

2 Single Layer Perceptron Implementation

2.1 Python Code

```
import pandas as pd
import numpy as np
from google.colab import files

uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)

X = df[['study_hours','attendance','assignment']].values
y = df['target'].values


def sigmoid(z):
    return 1/(1+np.exp(-z))


def sigmoid_derivative(z):
    s = sigmoid(z)
    return s*(1-s)


w = np.random.randn(3)
b = 0
learning_rate = 0.01

for epoch in range(100):
    total_loss = 0
    for i in range(len(X)):
        x = X[i]
        target = y[i]


        z = np.dot(x,w)+b
        a = sigmoid(z)


        error = a-target
        loss = error**2
        total_loss += loss
```

```
        dL_da = 2*error
        da_dz = sigmoid_derivative(z)
        dL_dz = dL_da * da_dz

        dL_dw = dL_dz*x
        dL_db = dL_dz


        w = w - learning_rate*dL_dw
        b = b - learning_rate*dL_db

    if epoch % 10 == 0:
        print("Epoch:",epoch,"Loss:",total_loss)

print("Final weight:",w)
print("Final bias:",b)
```

2.2 Output

```
student.csv
student.csv(text/csv) - 113 bytes, last modified: 18/06/2026 - 100% done
Saving student.csv to student.csv
[[ 2 60  1]
 [ 3 70  2]
 [ 5 80  3]
 [ 6 90  4]
 [ 1 50  1]
 [ 8 95  5]
 [ 4 75  3]
 [ 2 55  1]]
[0 0 1 1 0 1 1 0]
Epoch: 0 Loss: 4.0
Epoch: 10 Loss: 4.0
Epoch: 20 Loss: 4.0
Epoch: 30 Loss: 4.0
Epoch: 40 Loss: 4.0
Epoch: 50 Loss: 4.0
Epoch: 60 Loss: 4.0
Epoch: 70 Loss: 4.0
Epoch: 80 Loss: 4.0
Epoch: 90 Loss: 4.0
Final weight: [ 2.35404799  1.99468896 -1.69297389]
Final bias: 0.0
```